```
-- ######################################################################
-- Red social para la comunidad estudiantil pascualina
-- Script DML: consultas analíticas y vistas        (SGBD: PostgreSQL)
-- ######################################################################
--
-- REGLAS DE NOMENCLATURA PARA ALIAS (datos calculados)
--   * snake_case, en minúscula, sin tildes ni espacios.
--   * total_*     -> conteos (COUNT / SUM)          ej.
total_publicaciones
--   * promedio_*  -> promedios (AVG)                ej. promedio_semestre
--   * pct_*       -> porcentajes de 0 a 100         ej. pct_asistencia
--   * *_maximo / *_minimo -> MAX / MIN
--   * nombre_completo, edad, extracto -> datos derivados de texto/fechas
--   * Todo dato calculado lleva AS explícito.
-- ######################################################################

SET search_path TO red_social;

INSERT INTO estudiante (nombre, apellido, correo_institucional, carrera,
semestre, fecha_nacimiento, biografia) VALUES
 ('Marta',     'Rojas',    'marta.rojas@pascual.edu.co',     'Ingeniería de
Sistemas', 7, '2002-11-20', 'Apasionada por las bases de datos'),   -- id 3
 ('Carlos',    'Ruiz',     'carlos.ruiz@pascual.edu.co',     'Ingeniería de
Sistemas', 2, '2006-01-15', NULL),                                  -- id 4
 ('Sofía',     'Martínez', 'sofia.martinez@pascual.edu.co',  'Diseño
Gráfico',         8, '2001-05-30', 'Diseñadora UX y
mentora'),              -- id 5
 ('Andrés',    'Torres',   'andres.torres@pascual.edu.co',   'Administración
',         4, '2004-09-09', NULL),                                  -- id 6
 ('Valentina', 'López',    'valentina.lopez@pascual.edu.co', 'Ingeniería de
Sistemas', 6, '2003-02-27', NULL),                                  -- id 7
 ('Juan',      'Díaz',     'juan.diaz@pascual.edu.co',       'Derecho',
          1, '2007-06-18', NULL);                                   -- id 8
(sin actividad)

INSERT INTO habilidad (nombre, categoria) VALUES
 ('Python', 'Programación'), ('SQL', 'Programación'), ('Diseño UX',
'Diseño'),
 ('Liderazgo', 'Blandas'),   ('Oratoria', 'Blandas');

INSERT INTO interes (nombre, categoria) VALUES
 ('Inteligencia artificial', 'Tecnología'), ('Videojuegos', 'Ocio'),
 ('Emprendimiento', 'Negocios'),            ('Música', 'Arte');

INSERT INTO tiene_habilidad (id_estudiante, id_habilidad, nivel) VALUES
```

```
 (1,1,'avanzado'),(1,2,'intermedio'),(2,3,'avanzado'),(3,1,'avanzado'),(3,2,
'avanzado'),
 (3,4,'intermedio'),(4,1,'basico'),(5,3,'avanzado'),(5,5,'intermedio'),(6,4,
'avanzado'),
 (7,2,'intermedio'),(7,1,'intermedio');

INSERT INTO tiene_interes (id_estudiante, id_interes) VALUES
 (1,1),(1,3),(2,4),(3,1),(3,3),(4,1),(4,2),(5,4),(6,3),(7,1);

INSERT INTO publicacion (id_estudiante, contenido, tipo, fecha_hora) VALUES
 (3, 'Comparto una guía de índices en PostgreSQL',      'recurso',  now() -
interval '40 days'),  -- id 2
 (1, 'Hackathon de IA el próximo mes, ¿quién se une?',  'noticia',  now() -
interval '20 days'),  -- id 3
 (5, 'Meme: cuando el query corre en 0.01 s',           'meme',     now() -
interval '12 days'),  -- id 4
 (7, '¿Cómo normalizo una tabla a 3FN?',                'pregunta', now() -
interval '5 days'),   -- id 5
 (3, 'Tutorial de vistas materializadas',               'recurso',  now() -
interval '2 days'),   -- id 6
 (6, 'Feria de emprendimiento en el campus',            'noticia',  now() -
interval '1 day');    -- id 7

-- Un INSERT por comentario: el trigger asigna id_comentario (1, 2, 3...)
INSERT INTO comentario (id_publicacion, id_estudiante, contenido) VALUES (2,
1, 'Excelente aporte, justo lo que necesitaba.');
INSERT INTO comentario (id_publicacion, id_estudiante, contenido) VALUES (2,
4, '¿Incluye ejemplos con EXPLAIN?');
INSERT INTO comentario (id_publicacion, id_estudiante, contenido) VALUES (2,
7, 'Gracias por compartir.');
INSERT INTO comentario (id_publicacion, id_estudiante, contenido) VALUES (3,
3, 'Cuenten conmigo, puedo liderar el equipo.');
INSERT INTO comentario (id_publicacion, id_estudiante, contenido) VALUES (3,
7, 'Yo me anoto, ¿qué lenguaje usaremos?');
INSERT INTO comentario (id_publicacion, id_estudiante, contenido) VALUES (4,
2, 'Jajaja, muy bueno.');
INSERT INTO comentario (id_publicacion, id_estudiante, contenido) VALUES (4,
4, 'Total, me identifico.');
INSERT INTO comentario (id_publicacion, id_estudiante, contenido) VALUES (5,
3, 'Revisa dependencias transitivas y separa en tablas.');
INSERT INTO comentario (id_publicacion, id_estudiante, contenido) VALUES (5,
1, 'Empieza por identificar la clave y las dependencias funcionales.');
INSERT INTO comentario (id_publicacion, id_estudiante, contenido) VALUES (7,
1, 'Allí nos vemos.');
```

```sql
INSERT INTO reacciona (id_estudiante, id_publicacion, tipo_reaccion) VALUES
 (3,1,'me_gusta'),(7,1,'util'),
 (1,2,'util'),(2,2,'me_gusta'),(4,2,'util'),(7,2,'me_encanta'),
 (1,3,'me_encanta'),(3,3,'me_gusta'),(5,3,'me_gusta'),
 (1,4,'divertido'),(3,4,'divertido'),(2,4,'divertido'),(6,4,'me_gusta'),
 (3,5,'util'),(4,5,'util'),
 (1,7,'me_gusta');

INSERT INTO sigue (id_seguidor, id_seguido, es_mentoria) VALUES
 (4,3,TRUE),(7,3,FALSE),(1,3,FALSE),(5,2,FALSE),(3,1,FALSE),
 (4,1,FALSE),(6,1,FALSE),(1,5,FALSE),(2,5,TRUE);

INSERT INTO grupo (id_creador, nombre, descripcion, categoria) VALUES
 (3, 'Hackathon IA Pascual', 'Equipo para maratones de inteligencia
artificial', 'hackathon'), -- id 2
 (5, 'Club de Diseño',       'Espacio para compartir proyectos de
diseño',        'club'),      -- id 3
 (6, 'Emprendedores',        'Ideas de negocio y
networking',                    'club');      -- id 4

INSERT INTO participa_en (id_estudiante, id_grupo, rol) VALUES
 (3,1,'moderador'),(4,1,'miembro'),(7,1,'miembro'),
 (3,2,'administrador'),(1,2,'miembro'),(7,2,'miembro'),(4,2,'miembro'),
 (5,3,'administrador'),(2,3,'miembro'),
 (6,4,'administrador'),(1,4,'miembro');

INSERT INTO evento (id_organizador, id_grupo, nombre, fecha_hora_inicio,
fecha_hora_fin, ubicacion, tipo) VALUES
 (3, 2,    'Hackathon IA 2026',            now() + interval '10
days',                    now() + interval '10 days 8 hours',  'Auditorio
Central', 'taller'),  -- id 2
 (5, 3,    'Noche de integración',         now() + interval '5
days',                    now() + interval '5 days 3
hours',   'Cafetería',         'social'),  -- id 3
 (6, NULL, 'Feria de emprendimiento',      now() + interval '7
days',                    now() + interval '7 days 6
hours',   'Plazoleta',         'social'),  -- id 4 (independiente)
 (1, 1,    'Jornada de estudio de parciales', now() - interval '3
days',                  now() - interval '3 days' + interval '3 hours',
'Biblioteca', 'estudio'); -- id 5 (ya ocurrió)

INSERT INTO asiste (id_estudiante, id_evento, estado_asistencia) VALUES
 (1,1,'confirmado'),(3,1,'confirmado'),(7,1,'inscrito'),
 (3,2,'confirmado'),(1,2,'inscrito'),(7,2,'inscrito'),(4,2,'cancelado'),
 (5,3,'confirmado'),(2,3,'inscrito'),
```

```
 (6,4,'confirmado'),(1,4,'inscrito'),(4,4,'inscrito'),
 (1,5,'asistio'),(2,5,'asistio'),(4,5,'asistio'),(7,5,'cancelado');


-- ########################################################################
-- 1. CONSULTAS DE NEGOCIO (SELECT)
-- Cada consulta responde a una pregunta del modelo de negocio.
-- ########################################################################

-- CQ-01 ¿Quiénes conforman la comunidad y qué edad tienen?
-- Tablas: estudiante | Datos calculados: nombre_completo (texto), edad
(INTEGER)
SELECT e.id_estudiante,
       e.nombre || ' ' ||
e.apellido                                    AS nombre_completo,
       e.carrera,
       e.semestre,
       EXTRACT(YEAR FROM age(CURRENT_DATE,
e.fecha_nacimiento))::INTEGER   AS edad
FROM estudiante e
ORDER BY e.carrera, e.semestre DESC, nombre_completo;

-- CQ-02 ¿Cómo se distribuye la comunidad por carrera?
SELECT e.carrera,
       COUNT(*)                   AS total_estudiantes,
       ROUND(AVG(e.semestre), 1)   AS promedio_semestre,
       MIN(e.semestre)             AS semestre_minimo,
       MAX(e.semestre)             AS semestre_maximo
FROM estudiante e
GROUP BY e.carrera
ORDER BY total_estudiantes DESC, e.carrera;

-- CQ-03 ¿Cuáles son las 5 publicaciones con más interacción?
-- Se agregan comentarios y reacciones por separado (CTE) para no
multiplicar filas.
WITH comentarios_por_publicacion AS (
    SELECT id_publicacion, COUNT(*) AS total_comentarios
    FROM comentario
    GROUP BY id_publicacion
), reacciones_por_publicacion AS (
    SELECT id_publicacion, COUNT(*) AS total_reacciones
    FROM reacciona
    GROUP BY id_publicacion
)
SELECT p.id_publicacion,
```

```sql
       LEFT(p.contenido, 45)                                                AS
extracto,
       p.tipo                                                               AS
tipo_publicacion,
       COALESCE(c.total_comentarios, 0)                                     AS
total_comentarios,
       COALESCE(r.total_reacciones, 0)                                      AS
total_reacciones,
       COALESCE(c.total_comentarios, 0) + COALESCE(r.total_reacciones, 0) AS
engagement_total
FROM publicacion p
LEFT JOIN comentarios_por_publicacion c ON c.id_publicacion =
p.id_publicacion
LEFT JOIN reacciones_por_publicacion  r ON r.id_publicacion =
p.id_publicacion
ORDER BY engagement_total DESC, p.id_publicacion
LIMIT 5;

-- CQ-04 ¿Quiénes son los 5 estudiantes más activos?
-- Actividad = publicaciones + comentarios + reacciones dadas.
SELECT t.*,
       t.total_publicaciones + t.total_comentarios + t.total_reacciones AS
puntaje_actividad
FROM (
    SELECT e.id_estudiante,
           e.nombre || ' ' || e.apellido AS nombre_completo,
           (SELECT COUNT(*) FROM publicacion p WHERE p.id_estudiante =
e.id_estudiante) AS total_publicaciones,
           (SELECT COUNT(*) FROM comentario  c WHERE c.id_estudiante =
e.id_estudiante) AS total_comentarios,
           (SELECT COUNT(*) FROM reacciona   r WHERE r.id_estudiante =
e.id_estudiante) AS total_reacciones
    FROM estudiante e
) t
ORDER BY puntaje_actividad DESC, t.id_estudiante
LIMIT 5;

-- CQ-05 ¿Qué tipo de contenido se publica más?
SELECT p.tipo                                          AS
tipo_publicacion,
       COUNT(*)                                        AS
total_publicaciones,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1)  AS pct_del_total
FROM publicacion p
GROUP BY p.tipo
```

```sql
ORDER BY total_publicaciones DESC;

-- CQ-06 ¿Cómo reaccionan los estudiantes?
SELECT r.tipo_reaccion,
       COUNT(*)                                          AS
total_reacciones,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 1)  AS pct_del_total
FROM reacciona r
GROUP BY r.tipo_reaccion
ORDER BY total_reacciones DESC;

-- CQ-07 ¿Quiénes son los estudiantes con más seguidores y cuántos son
mentorías?
SELECT e.id_estudiante,
       e.nombre || ' ' || e.apellido      AS nombre_completo,
       COUNT(s.id_seguidor)               AS total_seguidores,
       COUNT(*) FILTER (WHERE s.es_mentoria) AS total_mentorias
FROM estudiante e
LEFT JOIN sigue s ON s.id_seguido = e.id_estudiante
GROUP BY e.id_estudiante
ORDER BY total_seguidores DESC, nombre_completo;

-- CQ-08 ¿Qué relaciones de mentoría existen? (autojoin de estudiante vía
sigue)
SELECT aprendiz.nombre || ' ' || aprendiz.apellido AS aprendiz,
       mentor.nombre   || ' ' || mentor.apellido   AS mentor,
       s.fecha_inicio                              AS fecha_inicio_mentoria,
       (CURRENT_DATE - s.fecha_inicio)             AS dias_de_mentoria
FROM sigue s
JOIN estudiante aprendiz ON aprendiz.id_estudiante = s.id_seguidor
JOIN estudiante mentor   ON mentor.id_estudiante   = s.id_seguido
WHERE s.es_mentoria
ORDER BY mentor, aprendiz;

-- CQ-09 ¿Qué habilidades son más comunes y con qué nivel promedio?
-- nivel_promedio: basico = 1, intermedio = 2, avanzado = 3
SELECT h.nombre                                    AS habilidad,
       h.categoria                                 AS categoria_habilidad,
       COUNT(*)                                    AS total_estudiantes,
       ROUND(AVG(CASE th.nivel WHEN 'basico'     THEN 1
                               WHEN 'intermedio' THEN 2
                               WHEN 'avanzado'   THEN 3 END), 2) AS
nivel_promedio,
       COUNT(*) FILTER (WHERE th.nivel = 'avanzado') AS total_avanzados
FROM habilidad h
```

```sql
JOIN tiene_habilidad th ON th.id_habilidad = h.id_habilidad
GROUP BY h.id_habilidad
ORDER BY total_estudiantes DESC, habilidad;

-- CQ-10 ¿Quién puede ser mentor en cada habilidad? (nivel avanzado,
ordenado por popularidad)
SELECT h.nombre                          AS habilidad,
       e.nombre || ' ' || e.apellido     AS posible_mentor,
       e.semestre,
       (SELECT COUNT(*) FROM sigue s WHERE s.id_seguido = e.id_estudiante)
AS total_seguidores
FROM tiene_habilidad th
JOIN habilidad  h ON h.id_habilidad   = th.id_habilidad
JOIN estudiante e ON e.id_estudiante  = th.id_estudiante
WHERE th.nivel = 'avanzado'
ORDER BY habilidad, total_seguidores DESC;

-- CQ-11 ¿Qué pares de estudiantes comparten 2 o más intereses? (autojoin,
a.id < b.id evita duplicados)
SELECT ea.nombre || ' ' || ea.apellido     AS estudiante_a,
       eb.nombre || ' ' || eb.apellido     AS estudiante_b,
       COUNT(*)                            AS total_intereses_comunes,
       STRING_AGG(i.nombre, ', ' ORDER BY i.nombre) AS intereses_comunes
FROM tiene_interes a
JOIN tiene_interes b ON b.id_interes = a.id_interes AND a.id_estudiante <
b.id_estudiante
JOIN interes      i  ON i.id_interes = a.id_interes
JOIN estudiante   ea ON ea.id_estudiante = a.id_estudiante
JOIN estudiante   eb ON eb.id_estudiante = b.id_estudiante
GROUP BY ea.id_estudiante, eb.id_estudiante
HAVING COUNT(*) >= 2
ORDER BY total_intereses_comunes DESC;

-- CQ-12 ¿Qué tan grandes son los grupos y cómo se reparten los roles?
SELECT g.id_grupo,
       g.nombre                                AS nombre_grupo,
       g.categoria                             AS categoria_grupo,
       cre.nombre || ' ' || cre.apellido       AS creador,
       COUNT(pe.id_estudiante)                 AS total_miembros,
       COUNT(*) FILTER (WHERE pe.rol = 'moderador')     AS
total_moderadores,
       COUNT(*) FILTER (WHERE pe.rol = 'administrador') AS
total_administradores
FROM grupo g
JOIN estudiante cre ON cre.id_estudiante = g.id_creador
```

```sql
LEFT JOIN participa_en pe ON pe.id_grupo = g.id_grupo
GROUP BY g.id_grupo, cre.id_estudiante
ORDER BY total_miembros DESC, nombre_grupo;

-- CQ-13 ¿Qué eventos vienen y cuántas inscripciones activas tienen?
-- Las inscripciones canceladas no cuentan. NULL en id_grupo = evento
independiente.
SELECT ev.id_evento,
       ev.nombre                                    AS nombre_evento,
       ev.tipo                                      AS tipo_evento,
       ev.fecha_hora_inicio,
       ev.ubicacion,
       org.nombre || ' ' || org.apellido            AS organizador,
       COALESCE(g.nombre, 'Evento independiente') AS grupo_asociado,
       COUNT(a.id_estudiante) FILTER (WHERE a.estado_asistencia <>
'cancelado') AS total_inscritos_activos
FROM evento ev
JOIN estudiante org ON org.id_estudiante = ev.id_organizador
LEFT JOIN grupo  g  ON g.id_grupo        = ev.id_grupo
LEFT JOIN asiste a  ON a.id_evento       = ev.id_evento
WHERE ev.fecha_hora_inicio > now()
GROUP BY ev.id_evento, org.id_estudiante, g.id_grupo
ORDER BY ev.fecha_hora_inicio;

-- CQ-14 ¿Cuál fue la tasa de asistencia de los eventos ya realizados?
SELECT ev.id_evento,
       ev.nombre                                                AS
nombre_evento,
       COUNT(*)                                                 AS
total_registros,
       COUNT(*) FILTER (WHERE a.estado_asistencia = 'asistio')     AS
total_asistieron,
       COUNT(*) FILTER (WHERE a.estado_asistencia = 'cancelado')   AS
total_cancelados,
       ROUND(100.0 * COUNT(*) FILTER (WHERE a.estado_asistencia = 'asistio')
/ COUNT(*), 1) AS pct_asistencia
FROM evento ev
JOIN asiste a ON a.id_evento = ev.id_evento
WHERE ev.fecha_hora_fin < now()
GROUP BY ev.id_evento
ORDER BY pct_asistencia DESC;

-- CQ-15 ¿Qué estudiantes no han tenido ninguna actividad? (NOT EXISTS)
SELECT e.id_estudiante,
       e.nombre || ' ' || e.apellido AS nombre_completo,
```

```sql
       e.carrera,
       e.semestre
FROM estudiante e
WHERE NOT EXISTS (SELECT 1 FROM publicacion p WHERE p.id_estudiante =
e.id_estudiante)
  AND NOT EXISTS (SELECT 1 FROM comentario  c WHERE c.id_estudiante =
e.id_estudiante)
  AND NOT EXISTS (SELECT 1 FROM reacciona   r WHERE r.id_estudiante =
e.id_estudiante)
ORDER BY nombre_completo;

-- CQ-16 ¿Qué publicaciones no han recibido respuesta? (LEFT JOIN + IS NULL)
SELECT p.id_publicacion,
       e.nombre || ' ' || e.apellido          AS autor,
       LEFT(p.contenido, 45)                  AS extracto,
       p.tipo                                 AS tipo_publicacion,
       (now()::DATE - p.fecha_hora::DATE)     AS dias_sin_respuesta
FROM publicacion p
JOIN estudiante e ON e.id_estudiante = p.id_estudiante
LEFT JOIN comentario c ON c.id_publicacion = p.id_publicacion
WHERE c.id_publicacion IS NULL
ORDER BY dias_sin_respuesta DESC;

-- CQ-17 ¿Cómo evoluciona la publicación de contenido mes a mes?
SELECT TO_CHAR(DATE_TRUNC('month', p.fecha_hora), 'YYYY-MM') AS mes,
       COUNT(*)                                              AS
total_publicaciones,
       COUNT(DISTINCT p.id_estudiante)                       AS
total_autores_activos
FROM publicacion p
GROUP BY DATE_TRUNC('month', p.fecha_hora)
ORDER BY DATE_TRUNC('month', p.fecha_hora);

-- CQ-18 Recomendación: compañeros de grupo que aún no se siguen
SELECT ea.nombre || ' ' || ea.apellido AS estudiante,
       eb.nombre || ' ' || eb.apellido AS sugerencia_seguir,
       COUNT(*)                        AS total_grupos_en_comun
FROM participa_en pa
JOIN participa_en pb ON pb.id_grupo = pa.id_grupo AND pb.id_estudiante <>
pa.id_estudiante
JOIN estudiante ea ON ea.id_estudiante = pa.id_estudiante
JOIN estudiante eb ON eb.id_estudiante = pb.id_estudiante
WHERE NOT EXISTS (SELECT 1 FROM sigue s
                  WHERE s.id_seguidor = pa.id_estudiante
                    AND s.id_seguido  = pb.id_estudiante)
```

```sql
GROUP BY ea.id_estudiante, eb.id_estudiante
ORDER BY total_grupos_en_comun DESC, estudiante, sugerencia_seguir
LIMIT 15;


-- ############################################################################
-- 2. VISTAS PARA ANALÍTICA
-- Convención: prefijo vw_ + nombre en singular/plural descriptivo.
-- Se eliminan en orden inverso de dependencia para que el script sea
-- re-ejecutable (idempotente).
-- ############################################################################

DROP VIEW IF EXISTS vw_estudiantes_inactivos;
DROP VIEW IF EXISTS vw_resumen_carrera;
DROP VIEW IF EXISTS vw_eventos_asistencia;
DROP VIEW IF EXISTS vw_resumen_grupo;
DROP VIEW IF EXISTS vw_afinidad_intereses;
DROP VIEW IF EXISTS vw_habilidades_estudiante;
DROP VIEW IF EXISTS vw_red_mentoria;
DROP VIEW IF EXISTS vw_engagement_publicacion;
DROP VIEW IF EXISTS vw_perfil_estudiante;

-- VW-01 Perfil 360° del estudiante (base de varias vistas posteriores)
-- Tipos: conteos -> BIGINT (COUNT), edad -> INTEGER, nombre_completo ->
TEXT
CREATE VIEW vw_perfil_estudiante AS
SELECT e.id_estudiante,
       e.nombre || ' ' || e.apellido                                    AS
nombre_completo,
       e.correo_institucional,
       e.carrera,
       e.semestre,
       EXTRACT(YEAR FROM age(CURRENT_DATE, e.fecha_nacimiento))::INTEGER AS
edad,
       (SELECT COUNT(*) FROM publicacion     x WHERE x.id_estudiante =
e.id_estudiante) AS total_publicaciones,
       (SELECT COUNT(*) FROM comentario      x WHERE x.id_estudiante =
e.id_estudiante) AS total_comentarios,
       (SELECT COUNT(*) FROM reacciona       x WHERE x.id_estudiante =
e.id_estudiante) AS total_reacciones_dadas,
       (SELECT COUNT(*) FROM sigue           x WHERE x.id_seguido    =
e.id_estudiante) AS total_seguidores,
       (SELECT COUNT(*) FROM sigue           x WHERE x.id_seguidor   =
e.id_estudiante) AS total_seguidos,
```

```
       (SELECT COUNT(*) FROM tiene_habilidad x WHERE x.id_estudiante =
e.id_estudiante) AS total_habilidades,
       (SELECT COUNT(*) FROM tiene_interes   x WHERE x.id_estudiante =
e.id_estudiante) AS total_intereses,
       (SELECT COUNT(*) FROM participa_en    x WHERE x.id_estudiante =
e.id_estudiante) AS total_grupos
FROM estudiante e;

-- VW-02 Interacción por publicación (comentarios + reacciones)
CREATE VIEW vw_engagement_publicacion AS
SELECT p.id_publicacion,
       p.id_estudiante                  AS id_autor,
       e.nombre || ' ' || e.apellido    AS autor,
       p.tipo                           AS tipo_publicacion,
       p.fecha_hora                     AS fecha_publicacion,
       LEFT(p.contenido, 60)            AS extracto,
       COALESCE(c.total, 0)             AS total_comentarios,
       COALESCE(r.total, 0)             AS total_reacciones,
       COALESCE(c.total, 0) + COALESCE(r.total, 0) AS engagement_total
FROM publicacion p
JOIN estudiante e ON e.id_estudiante = p.id_estudiante
LEFT JOIN (SELECT id_publicacion, COUNT(*) AS total FROM comentario GROUP BY
id_publicacion) c
       ON c.id_publicacion = p.id_publicacion
LEFT JOIN (SELECT id_publicacion, COUNT(*) AS total FROM reacciona  GROUP BY
id_publicacion) r
       ON r.id_publicacion = p.id_publicacion;

-- VW-03 Red de mentorías (aprendiz -> mentor)
CREATE VIEW vw_red_mentoria AS
SELECT s.id_seguidor                        AS id_aprendiz,
       aprendiz.nombre || ' ' || aprendiz.apellido AS aprendiz,
       s.id_seguido                         AS id_mentor,
       mentor.nombre   || ' ' || mentor.apellido   AS mentor,
       s.fecha_inicio                       AS fecha_inicio_mentoria,
       (CURRENT_DATE - s.fecha_inicio)      AS dias_de_mentoria
FROM sigue s
JOIN estudiante aprendiz ON aprendiz.id_estudiante = s.id_seguidor
JOIN estudiante mentor   ON mentor.id_estudiante   = s.id_seguido
WHERE s.es_mentoria;

-- VW-04 Habilidades de cada estudiante con nivel numérico para poder
promediar
CREATE VIEW vw_habilidades_estudiante AS
SELECT e.id_estudiante,
```

```
       e.nombre || ' ' || e.apellido AS nombre_completo,
       e.carrera,
       h.nombre                      AS habilidad,
       h.categoria                   AS categoria_habilidad,
       th.nivel,
       CASE th.nivel WHEN 'basico'     THEN 1
                     WHEN 'intermedio' THEN 2
                     WHEN 'avanzado'   THEN 3 END AS nivel_numerico
FROM tiene_habilidad th
JOIN estudiante e ON e.id_estudiante = th.id_estudiante
JOIN habilidad  h ON h.id_habilidad  = th.id_habilidad;

-- VW-05 Pares de estudiantes con intereses en común (base de
recomendaciones)
CREATE VIEW vw_afinidad_intereses AS
SELECT a.id_estudiante                       AS id_estudiante_a,
       ea.nombre || ' ' || ea.apellido       AS estudiante_a,
       b.id_estudiante                       AS id_estudiante_b,
       eb.nombre || ' ' || eb.apellido       AS estudiante_b,
       COUNT(*)                              AS total_intereses_comunes,
       STRING_AGG(i.nombre, ', ' ORDER BY i.nombre) AS intereses_comunes
FROM tiene_interes a
JOIN tiene_interes b ON b.id_interes = a.id_interes AND a.id_estudiante <
b.id_estudiante
JOIN interes     i  ON i.id_interes  = a.id_interes
JOIN estudiante  ea ON ea.id_estudiante = a.id_estudiante
JOIN estudiante  eb ON eb.id_estudiante = b.id_estudiante
GROUP BY a.id_estudiante, ea.nombre, ea.apellido, b.id_estudiante,
eb.nombre, eb.apellido;

-- VW-06 Resumen de grupos: tamaño, roles y eventos asociados
CREATE VIEW vw_resumen_grupo AS
SELECT g.id_grupo,
       g.nombre                         AS nombre_grupo,
       g.categoria                      AS categoria_grupo,
       g.fecha_creacion,
       cre.nombre || ' ' || cre.apellido AS creador,
       COALESCE(m.total_miembros, 0)         AS total_miembros,
       COALESCE(m.total_moderadores, 0)      AS total_moderadores,
       COALESCE(m.total_administradores, 0)  AS total_administradores,
       COALESCE(ev.total_eventos, 0)         AS total_eventos
FROM grupo g
JOIN estudiante cre ON cre.id_estudiante = g.id_creador
LEFT JOIN (SELECT id_grupo,
                  COUNT(*) AS total_miembros,
```

```
                COUNT(*) FILTER (WHERE rol = 'moderador')     AS
total_moderadores,
                COUNT(*) FILTER (WHERE rol = 'administrador') AS
total_administradores
          FROM participa_en GROUP BY id_grupo) m ON m.id_grupo = g.id_grupo
LEFT JOIN (SELECT id_grupo, COUNT(*) AS total_eventos
          FROM evento WHERE id_grupo IS NOT NULL GROUP BY id_grupo) ev ON
ev.id_grupo = g.id_grupo;

-- VW-07 Eventos con estado temporal, inscripciones y asistencia
CREATE VIEW vw_eventos_asistencia AS
SELECT ev.id_evento,
       ev.nombre                                    AS nombre_evento,
       ev.tipo                                      AS tipo_evento,
       ev.fecha_hora_inicio,
       ev.fecha_hora_fin,
       CASE WHEN ev.fecha_hora_fin    < now() THEN 'finalizado'
            WHEN ev.fecha_hora_inicio > now() THEN 'programado'
            ELSE 'en_curso' END                     AS estado_evento,
       ev.ubicacion,
       org.nombre || ' ' || org.apellido            AS organizador,
       COALESCE(g.nombre, 'Evento independiente') AS grupo_asociado,
       COUNT(a.id_estudiante)                                          AS
total_registros,
       COUNT(*) FILTER (WHERE a.estado_asistencia = 'confirmado')   AS
total_confirmados,
       COUNT(*) FILTER (WHERE a.estado_asistencia = 'asistio')      AS
total_asistieron,
       COUNT(*) FILTER (WHERE a.estado_asistencia = 'cancelado')    AS
total_cancelados,
       ROUND(100.0 * COUNT(*) FILTER (WHERE a.estado_asistencia = 'asistio')
            / NULLIF(COUNT(a.id_estudiante), 0), 1)                 AS
pct_asistencia
FROM evento ev
JOIN estudiante org ON org.id_estudiante = ev.id_organizador
LEFT JOIN grupo  g  ON g.id_grupo  = ev.id_grupo
LEFT JOIN asiste a  ON a.id_evento = ev.id_evento
GROUP BY ev.id_evento, org.id_estudiante, g.id_grupo;

-- VW-08 Resumen por carrera (construida sobre vw_perfil_estudiante)
CREATE VIEW vw_resumen_carrera AS
SELECT p.carrera,
       COUNT(*)                             AS total_estudiantes,
       ROUND(AVG(p.semestre), 1)            AS promedio_semestre,
       SUM(p.total_publicaciones)::INTEGER  AS total_publicaciones,
```

```
       SUM(p.total_comentarios)::INTEGER     AS total_comentarios,
       SUM(p.total_reacciones_dadas)::INTEGER AS total_reacciones_dadas,
       ROUND(AVG(p.total_seguidores), 2)     AS promedio_seguidores
FROM vw_perfil_estudiante p
GROUP BY p.carrera;

-- VW-09 Estudiantes sin actividad (útil para campañas de reactivación)
CREATE VIEW vw_estudiantes_inactivos AS
SELECT p.id_estudiante,
       p.nombre_completo,
       p.correo_institucional,
       p.carrera,
       p.semestre,
       p.total_grupos,
       p.total_intereses
FROM vw_perfil_estudiante p
WHERE p.total_publicaciones + p.total_comentarios + p.total_reacciones_dadas
= 0;


-- ########################################################################
-- 3. CONSULTAS QUE REUTILIZAN LAS VISTAS
-- ########################################################################

-- VQ-01 Perfiles más completos (ranking por seguidores)
SELECT nombre_completo, carrera, total_seguidores, total_publicaciones,
total_habilidades
FROM vw_perfil_estudiante
ORDER BY total_seguidores DESC, total_publicaciones DESC
LIMIT 5;

-- VQ-02 Top 3 publicaciones por interacción
SELECT id_publicacion, autor, tipo_publicacion, extracto, engagement_total
FROM vw_engagement_publicacion
ORDER BY engagement_total DESC
LIMIT 3;

-- VQ-03 Comparativo entre carreras
SELECT carrera, total_estudiantes, promedio_semestre, total_publicaciones,
promedio_seguidores
FROM vw_resumen_carrera
ORDER BY total_publicaciones DESC;

-- VQ-04 Mentores con más aprendices
```

```sql
SELECT mentor, COUNT(*) AS total_aprendices, MIN(fecha_inicio_mentoria) AS
primera_mentoria
FROM vw_red_mentoria
GROUP BY mentor
ORDER BY total_aprendices DESC;

-- VQ-05 Estudiantes con SQL avanzado o intermedio (candidatos a tutores)
SELECT nombre_completo, carrera, nivel
FROM vw_habilidades_estudiante
WHERE habilidad = 'SQL' AND nivel_numerico >= 2
ORDER BY nivel_numerico DESC, nombre_completo;

-- VQ-06 Mejores coincidencias de intereses
SELECT estudiante_a, estudiante_b, total_intereses_comunes,
intereses_comunes
FROM vw_afinidad_intereses
ORDER BY total_intereses_comunes DESC;

-- VQ-07 Grupos más grandes con sus eventos
SELECT nombre_grupo, categoria_grupo, total_miembros, total_eventos
FROM vw_resumen_grupo
ORDER BY total_miembros DESC;

-- VQ-08 Agenda: eventos programados con inscripciones
SELECT nombre_evento, fecha_hora_inicio, grupo_asociado, total_registros -
total_cancelados AS total_inscritos_activos
FROM vw_eventos_asistencia
WHERE estado_evento = 'programado'
ORDER BY fecha_hora_inicio;

-- VQ-09 Estudiantes a reactivar
SELECT nombre_completo, correo_institucional, carrera
FROM vw_estudiantes_inactivos;

-- VQ-10 Función de ventana: posición de cada estudiante dentro de su
carrera
SELECT carrera, nombre_completo, total_publicaciones,
       RANK() OVER (PARTITION BY carrera ORDER BY total_publicaciones DESC)
AS posicion_en_carrera
FROM vw_perfil_estudiante
ORDER BY carrera, posicion_en_carrera;

-- VQ-11 Composición de vistas: interacción promedio de las publicaciones
por carrera del autor
SELECT p.carrera,
```

```sql
       COUNT(*)                          AS total_publicaciones,
       ROUND(AVG(ep.engagement_total), 2) AS promedio_engagement
FROM vw_engagement_publicacion ep
JOIN vw_perfil_estudiante p ON p.id_estudiante = ep.id_autor
GROUP BY p.carrera
ORDER BY promedio_engagement DESC;


-- ############################################################################
-- 4. DML TRANSACCIONAL (UPDATE / DELETE)
-- Se ejecuta dentro de una transacción y termina en ROLLBACK para no
-- alterar los datos de prueba.
-- ############################################################################
BEGIN;

-- Ascender a moderador a un miembro activo del grupo 1
UPDATE participa_en
SET    rol = 'moderador'
WHERE  id_estudiante = 4 AND id_grupo = 1;

-- Limpiar inscripciones canceladas del evento 2
DELETE FROM asiste
WHERE  estado_asistencia = 'cancelado' AND id_evento = 2;

-- Verificar el efecto sobre las vistas
SELECT nombre_grupo, total_moderadores FROM vw_resumen_grupo WHERE id_grupo
= 1;
SELECT nombre_evento, total_registros, total_cancelados FROM
vw_eventos_asistencia WHERE id_evento = 2;

ROLLBACK;   -- cambiar a COMMIT para confirmar los cambios
```